Звіт з лабораторної роботи 5

Побудова діаграми станів
класу Сповіщення

Виконав студент групи ІПЗ-23-3
Некрасов Артур Дмитрович

2026

1 Мета та вибір класу

Мета роботи — побудувати діаграму станів об’єкта «Сповіщення» для UC12 «Повідомити читача» університетської бібліотеки. Модель містить 7 станів, складений стан із двома послідовними підстанами, неглибоку історію, охоронні умови, дії входу й виходу та внутрішні переходи. Перевірено звичайне надсилання і відновлення після паузи з повтором після помилки.

Клас обрано з власної діаграми класів ЛР2. Він уже має state: MessageState, attempts і retryAt, а також send() та scheduleRetry(at). У ЛР4 повідомлення PENDING зберігається разом із бронюванням і передається на фонове опрацювання. Помилка доставки не скасовує бронювання. Така поведінка має розгалуження й тривалі очікування, тому її доцільно описати діаграмою станів.

Таблиця 1 — Відбір станів і подій

Джерело або властивість	Рішення для діаграми
state: MessageState PENDING, SENT, RETRY	PENDING деталізовано; SENT — приймання сервісом, RETRY — очікування повтору.
attempts: Integer = 0	Лічильник збільшується лише при вході в «Очікує відповіді».
retryAt: DateTime [0..1]	Охоронна умова повтору now ≥ retryAt.
send() та scheduleRetry(at)	Події запуску спроби й перепланування строку повтору.
Фоновий UC12 та сигнал A17 з ЛР4	Два послідовні підстани: «У черзі» → «Готове до спроби».
Уточнення проєкту в ЛР5	Пауза підготовки з поверненням через H; окрема фіксація відповіді сервісу.

Переглянуто всі чотири вкладення ЛР5: HTML і Markdown містять завдання, PlantUML і SVG — приклад для кур’єра. Власна модель продовжує предметну область бібліотеки. Нові операції та правило паузи є уточненням ЛР5; їх явно додано до фрагмента класу в розділі 6, без приписування початковій версії ЛР2.

2 Складений стан та неглибока історія

Для читабельності одну машину станів показано двома фрагментами. Стрілка T03 продовжується на наступному рисунку. Обидва рисунки є справжніми скріншотами відкритої моделі в draw.io; повний граф збережено у файлах PlantUML і SVG.

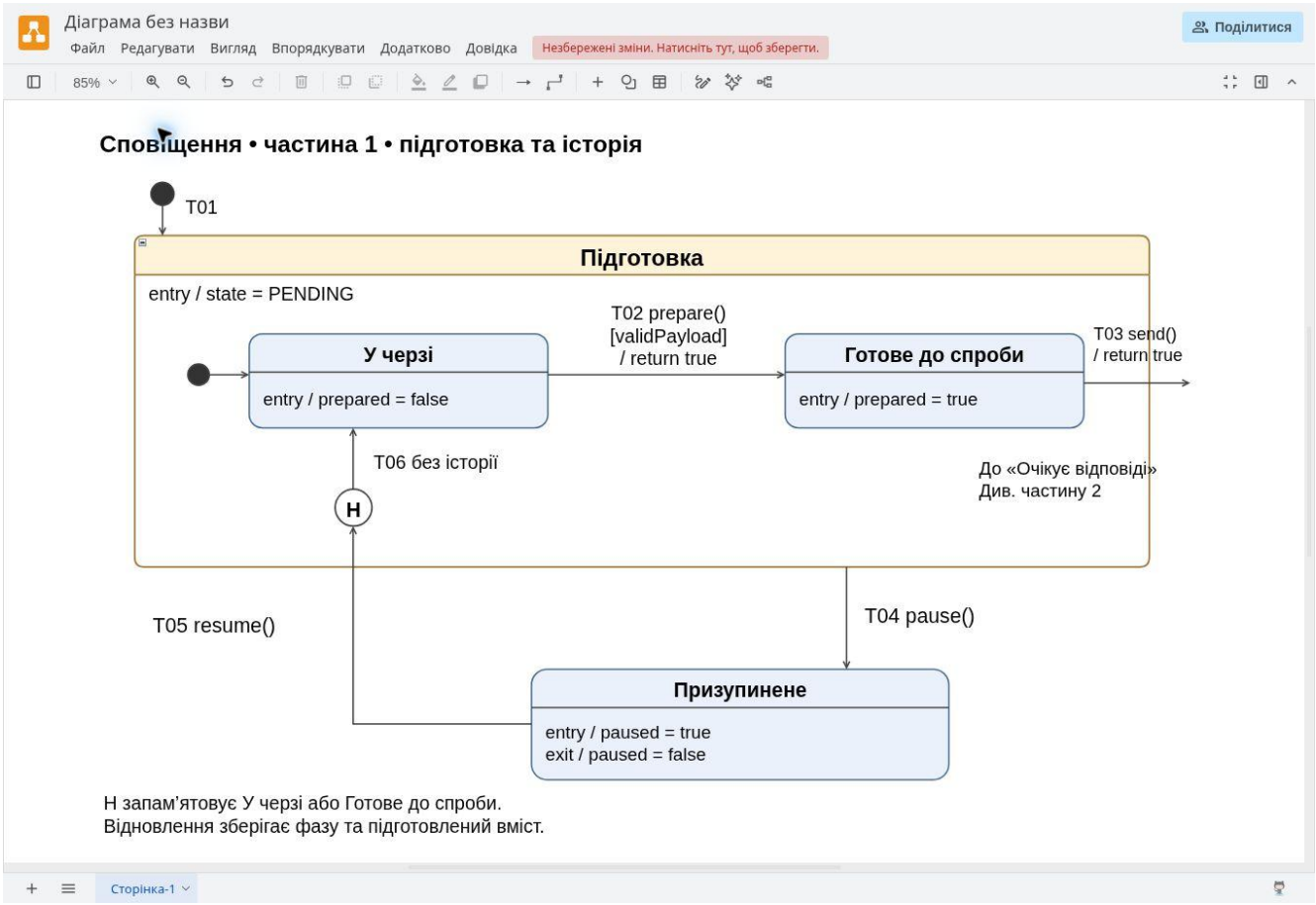


Рисунок 1 — Підготовка повідомлення та відновлення через H

У «Підготовці» об'єкт спочатку перебуває в «У черзі». prepare() за validPayload переводить його в «Готове до спроби»; validPayload означає непорожні recipient і text. Вміст після підготовки не змінюється. pause() виходить із межі складеного стану, тому дозволений з обох його підстанів.

resume() веде до H і відновлює останній активний підстан на цьому рівні вкладеності [3, 4]. Тому пауза до prepare() повертає в чергу, а пауза після prepare() — у готовність. Повторний вхід виконує entry відновленого стану, але не запускає надсилання й не збільшує attempts. T06 задає «У черзі» за відсутності історії; у звичайному resume() історія вже існує.

3 Надсилання та обробка результату

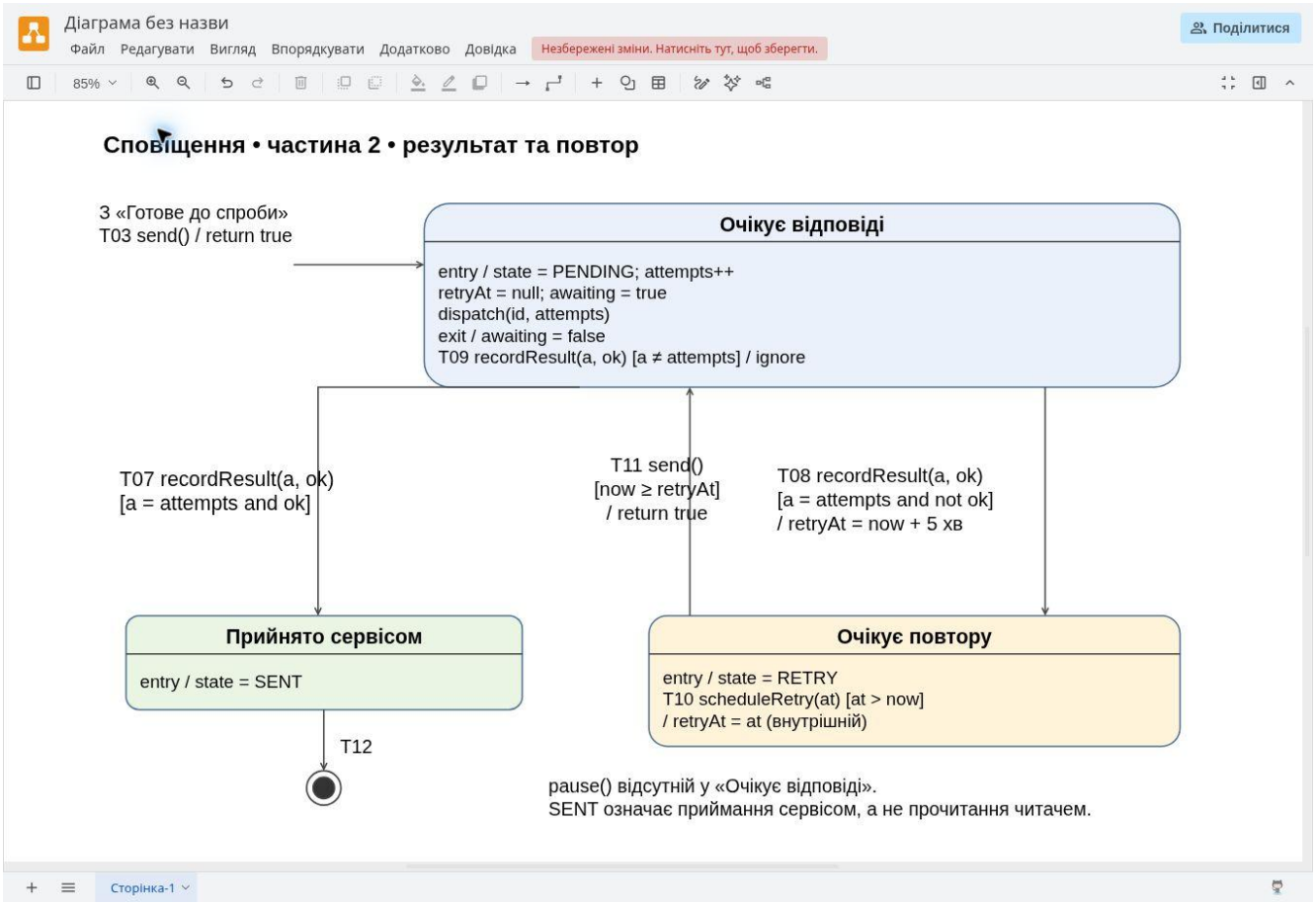


Рисунок 2 — Поточна спроба, повтор та завершення

При вході в «Очікує відповіді» збільшується attempts, очищується retryAt, установлюється awaiting і викликається неблокувальний dispatch(id, attempts). Адаптер подає результат через recordResult(a, ok); тайм-аут подається як ok = false. send() повертає true при запуску спроби, що не підтверджує приймання зовнішнім сервісом.

Результат поточної спроби з ok = true установлює SENT і завершує машину; запис залишається в БД. При ok = false встановлюється повтор через 5 хвилин. Це прийнятий у ЛР5 інтервал, який можна змінити scheduleRetry(at). Запізнілий результат із a ≠ attempts обробляється внутрішнім переходом T09.

T10 є внутрішнім переходом: змінює лише retryAt без виходу і повторного входу в стан [5]. У «Очікує відповіді» exit скидає awaiting. Усі події одного об’єкта опрацьовуються послідовно. Робота перевіряє модель; реальне надсилання листів, запуск бібліотечної БД та мережеві вимірювання не виконувалися.

4 Таблиця станів і переходів

Усього визначено 7 станів: складений «Підготовка», його підстани «У черзі» й «Готове до спроби» та прості «Призупинене», «Очікує відповіді», «Очікує повтору», «Прийнято сервісом». Отже, навіть без урахування складеного стану є 6 простих станів. Початкові вузли, H і кінцевий вузол до цього числа не входять.

Таблиця 2 — Початковий вхід, підготовка та історія

ID	З якого стану	Подія та охоронна умова	До якого стану та ефект
T01	Початок	—	Підготовка / У черзі Початковий вхід; attempts = 0.
T02	У черзі	prepare() [validPayload]	Готове до спроби entry Ready: prepared = true; повернути true.
T03	Готове до спроби	send()	Очікує відповіді Запустити нову спробу; повернути true.
T04	Підготовка	pause()	Призупинене Діє з обох підстанів; H запам’ятовує активний.
T05	Призупинене	resume()	Підготовка / H H відновлює У черзі або Готове до спроби.
T06	H без історії	—	У черзі Резервний початковий вибір.

Проєкція на статуси ЛР2

«Підготовка» та її підстани, «Призупинене» й «Очікує відповіді» мають state = PENDING. «Очікує повтору» має RETRY, а «Прийнято сервісом» — SENT. Нових значень MessageState не введено. Детальну фазу відображає похідний атрибут /phase: NotificationPhase; H зберігає підстан незалежно від поточного значення phase = PAUSED.

Пауза зупиняє лише опрацювання сповіщення. Строк бронювання expiresAt продовжує спливати; стан резерву та примірника ця машина не змінює. У цій лабораторній моделюється технічний життєвий цикл одного збереженого повідомлення, а не скасування чи продовження бронювання.

Продовження таблиці переходів

Таблиця 2 — Надсилання, повтор та кінцевий стан

ID	З якого стану	Подія та охоронна умова	До якого стану та ефект
T07	Очікує відповіді	recordResult(a, ok) [a = attempts and ok]	Прийнято сервісом entry Sent: state = SENT.
T08	Очікує відповіді	recordResult(a, ok) [a = attempts and not ok]	Очікує повтору retryAt = now + 5 хв; entry Retry: state = RETRY.
T09	Очікує відповіді	recordResult(a, ok) [a ≠ attempts]	Без зміни Внутрішній перехід; відкинути застарілий результат.
T10	Очікує повтору	scheduleRetry(at) [at > now]	Без зміни Внутрішній перехід; retryAt = at, без entry / exit.
T11	Очікує повтору	send() [now ≥ retryAt]	Очікує відповіді Повторити підготовлений вміст; повернути true.
T12	Прийнято сервісом	Завершення стану	Кінцевий псевдостан Автоматично; запис SENT залишається в БД.

Однозначність умов та обробка відмов

Для recordResult(a, ok) у стані очікування задано три взаємовиключні випадки: a = attempts і ok; a = attempts і not ok; a ≠ attempts. За Boolean ok рівно один випадок істинний. Охоронні умови лише читають параметри й атрибути. Їх оцінюють до виконання ефекту переходу [6].

Якщо validPayload хибне, prepare() повертає false й об’єкт залишається в черзі. Якщо строк повтору ще не настав, send() повертає false без нової спроби. Для інших подій без визначеного переходу прийнято політику: подія споживається, стан і дані не змінюються; Boolean-операції повертають false. Так само відхиляються виклики після завершення машини.

П’ятихвилинний інтервал, операції pause/resume і передача номера спроби є явно прийнятими проєктними уточненнями. Один об’єкт має не більше одного активного запиту. Модель сама не доводить одноразової фізичної доставки за мережевих збоїв; для цього шлюз повинен підтримувати ідемпотентність із постійним ключем id повідомлення.

5 Перевірка проходження

Послідовність А успішне надсилання

Початкові дані: attempts = 0, retryAt = null, заповнені recipient і text. Час наведено в межах одного умовного дня. Проміжний стан «Прийнято сервісом» завершується T12 автоматично в тому самому кроці.

Таблиця 3 — Проходження послідовності А

Крок і час	Подія	Стан після події	Результат
0 10:00	Створення	У черзі	T01 attempts = 0
1 10:00	prepare() [validPayload]	Готове до спроби	T02 attempts = 0
2 10:01	send()	Очікує відповіді	T03 attempts = 1
3 10:02	recordResult(1, true)	Кінець після SENT	T07+T12 attempts = 1

Навмисно відсутні переходи

Зі стану «Очікує відповіді» відсутній pause(). Запит уже передано зовнішньому сервісу й надійне скасування не передбачене. Об’єкт повинен дочекатися відповіді або тайм-ауту, щоб не втратити результат активної спроби. Також із цього стану немає send(): другу одночасну спробу запускати не можна. Ці заборони задає сама структура діаграми.

Таблиця 4 — Реакції на всі шість видів подій

Стан	prepare	send	recordResult	scheduleRetry	pause	resume
У черзі	T02	—	—	—	T04	—
Готове	—	T03	—	—	T04	—
Пауза	—	—	—	—	—	T05
Відповідь	—	—	T07 / T08 / T09	—	—	—
Повтор	—	T11	—	T10	—	—
Кінець	—	—	—	—	—	—

Послідовність Б паузи та повтор після помилки

Послідовність перевіряє обидва значення історії, заборону паузи під час запиту, перепланування та запізнілу відповідь. Усі не вказані в таблиці атрибути зберігаються.

Таблиця 5 — Проходження послідовності Б

Крок і час	Подія	Стан після події	Перехід та дані
0 10:00	Створення	У черзі	T01 спроб: 0
1 10:00	pause()	Призупинене	T04 спроб: 0
2 10:00	resume()	У черзі	T05 спроб: 0
3 10:00	prepare() [validPayload]	Готове до спроби	T02 спроб: 0
4 10:00	pause()	Призупинене	T04 спроб: 0
5 10:00	resume()	Готове до спроби	T05 спроб: 0
6 10:00	send()	Очікує відповіді	T03 спроб: 1
7 10:00	pause() — ігнорується	Очікує відповіді	Без переходу спроб: 1
8 10:00	recordResult(1, false)	Очікує повтору	T08 спроб: 1; строк 10:05
9 10:01	scheduleRetry(10:07)	Очікує повтору	T10 спроб: 1; строк 10:07
10 10:06	send() — зарано	Очікує повтору	Без переходу спроб: 1; строк 10:07
11 10:07	send()	Очікує відповіді	T11 спроб: 2
12 10:07	recordResult(1, true) — застаріла	Очікує відповіді	T09 спроб: 2
13 10:08	recordResult(2, true)	Кінець після SENT	T07+T12 спроб: 2

6 Узгодження з діаграмою класів

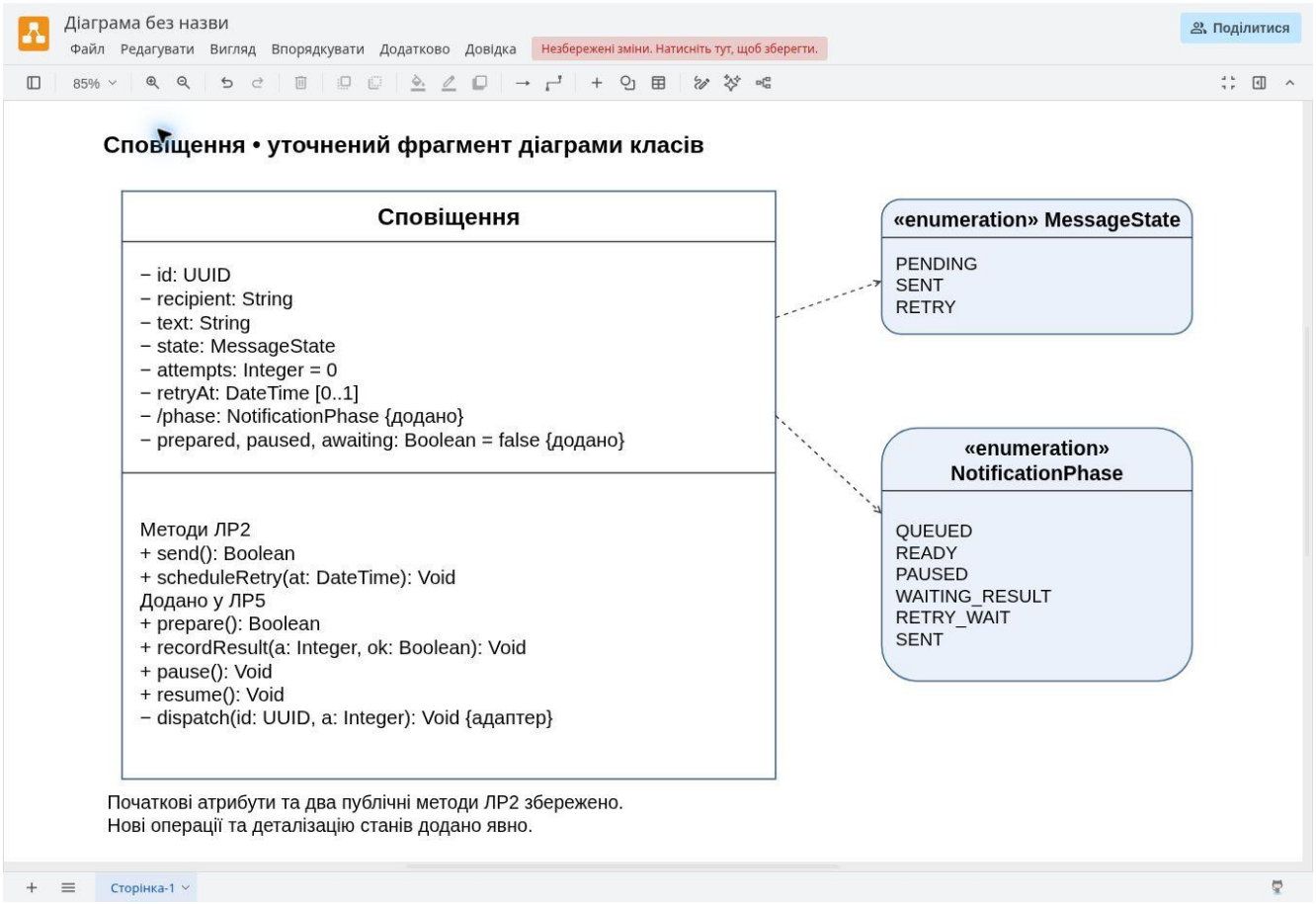


Рисунок 3 — Уточнений фрагмент класу Сповіщення

Перевірка початкової ЛР2 виявила лише два методи для цього класу: send() та scheduleRetry(at). Чотири інші події початково не були задекларовані. Тому для узгодження додано prepare(), recordResult(a, ok), pause() і resume() до уточненого фрагмента ЛР2, який входить до результатів ЛР5. Початковий звіт ЛР2 не змінено; відповідність повна саме після цього явного уточнення.

Нові атрибути prepared, paused та awaiting відображають дії входу/виходу. /phase є похідним від активного простого стану; його значення перелічені в NotificationPhase. Приватний dispatch() належить адаптеру, є дією входу, а не зовнішньою подією машини. Ініціалізація, резервний вихід H і завершення також не потребують публічних методів.

Висновок

Побудовано машину станів сповіщення з двома послідовними підстанами та коректним відновленням через H. Перевірено успіх, повтор, заборонені події та застарілий результат. Уточнення класу усунуло розбіжність між подіями й початковими методами. PlantUML успішно сформував SVG; до звіту включено три справжні скріншоти редактора.

Використані джерела

Методичні вказівки ЛР5: усі чотири надані файли HTML, Markdown, PlantUML і SVG.

Власні ЛР1–4: UC12, клас Сповіщення, фонове опрацювання підтвердження бронювання.

PlantUML. State Diagram. https://plantuml.com/state-diagram

IBM. Pseudostates in state machine diagrams. https://www.ibm.com/docs/en/dma?topic=machines-pseudostates

IBM. Designating a self transition. https://www.ibm.com/docs/en/bpm/8.5.6?topic=machine-designating-self-transition

IBM. Adding triggers to state machine diagrams. https://www.ibm.com/docs/en/dma?topic=states-adding-triggers